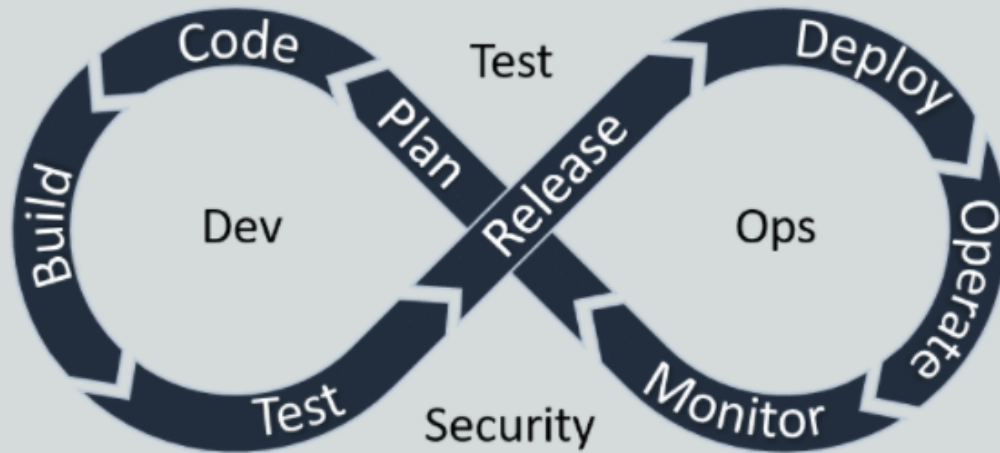


DevOps

Culture & Fundamentals



An infinity loop formed by the software lifecycle stages is a common depiction of the ongoing collaboration and efficiencies suggested by DevOps.

DevOps is short for Development (Dev) and Operations (Ops). Dev are the people and processes that create software. Ops are the teams and processes that deliver and monitor the software.

Developers change things quickly, release often, and measure success by the rate of delivery. Operations are driven by maintaining stability of the application. Frequent releases are a cause for concern of the stability and reliability of the application on the supported platforms, especially during high network traffic.

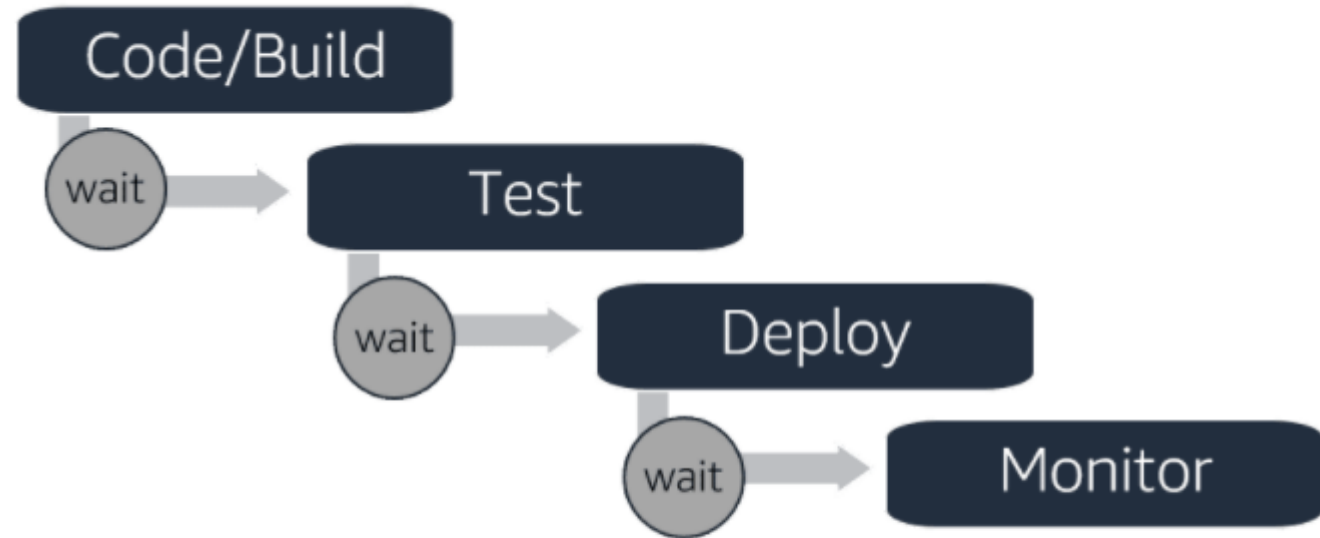
DevOps brings together formerly siloed roles (development, IT operations, quality engineering, and security) to optimize the productivity of developers and the reliability of operations.

DevOps is a combination of:

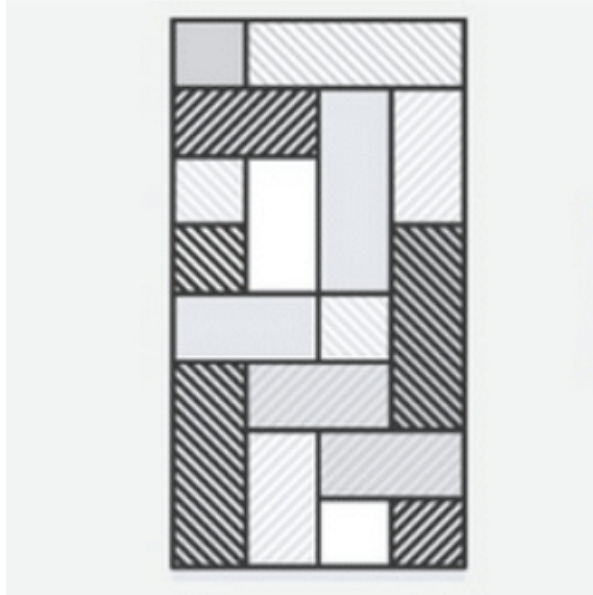
- Cultural philosophies for removing barriers and sharing end-to-end responsibility
- Processes developed for speed and quality, that streamline the way people work
- Tools that align with processes and automate repeatable tasks, making the release process more efficient and the application more reliable

As you go through the course, keep in mind that teams practice DevOps to increase innovation and agility, improve quality, release faster, and lower cost. DevOps improves delivery efficiency and predictability of the application and services.

- Waterfall development projects are slow, not iterative, resistant to change, and have long release cycles. Some reasons for this include:
 - Requirements are rigid, set at project start, and will likely not change.
 - Development phases are siloed, each starting after the previous phase has ended. Each phase is supported by highly specialized teams. Handoffs from one phase to the other are long, often requiring teams to switch tools and spend time clarifying incomplete or ambiguous information.
 - Testing and security come after implementation, making corrective actions responsive and expensive.



Development phases supported by siloed and specialized teams cause delays and are costly.



Monolithic applications have tightly coupled functionality.

- Monolithic applications are hard to update and deploy because they:
 - Are developed and deployed as a unit, so when changes are made, the entire application must be redeployed
 - Have tightly coupled functionality, so if the application is large enough, maintenance becomes an issue because developers have a hard time understanding the entire application
 - Are implemented using a single development stack, so changing technology is difficult and costly

- Manual processes throughout the application lifecycle are slow, inconsistent, and error-prone. For example, manually setting and configuring the infrastructure is time-consuming. Manually repeating this process is no guarantee that a step will not be missed. Another example is *telling* the developers to make sure their code is thoroughly tested before pushing it. Even with the best intentions, this manual process is slow, and does not preclude someone from forgetting a test or two.
- Teams supporting the software development lifecycle have traditionally been siloed. Specialized in their skill set, teams such as business, development, quality assurance, specialists, maintenance, and operations, have been separated from each other and require scheduled and rigid hand-offs. Even though these teams have a common goal of delivering and supporting the application, they also have their own priorities, tooling, and processes. It is difficult to achieve efficiencies when project members are reporting to different units and aimed for different targets.

106x

Faster from
commit to
deploy



208x

More frequent
deployments



7x

Lower change
failure rate



2,604x

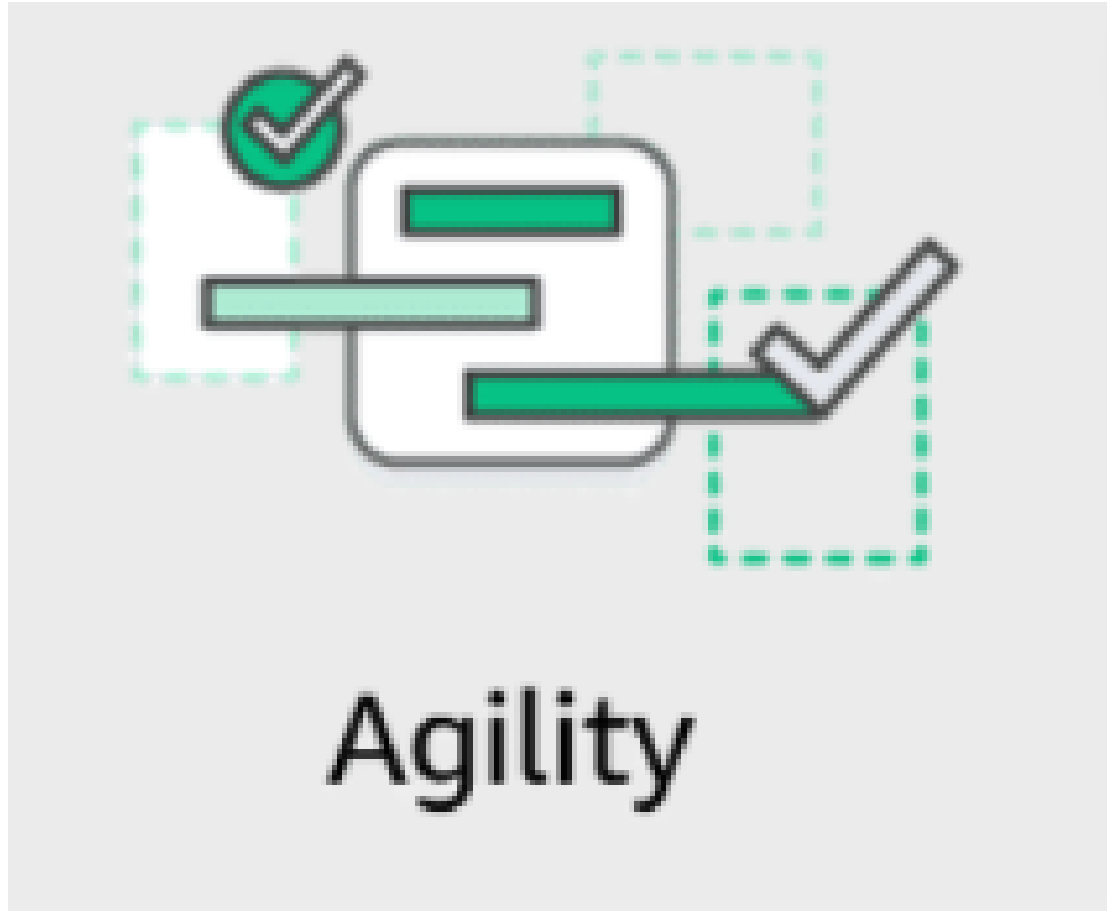
Faster time to
recover



Proven success

According to the [Accelerate State of DevOps 2019](#) report, organizations of any size, and in any industry, can benefit from DevOps. According to the report, teams who successfully adopt DevOps see shorter delivery cycles, decreased change failure rates, and improved performance.

Source: [Accelerate State of DevOps 2019](#), DevOps Research and Assessment (DORA)



Build a competitive advantage by anticipating market needs and quickly deliver value. Monitor your applications and support your development strategies with real data. Have development practices in place that allow teams to move at high velocity and continuously deliver controlled, small updates as needed.



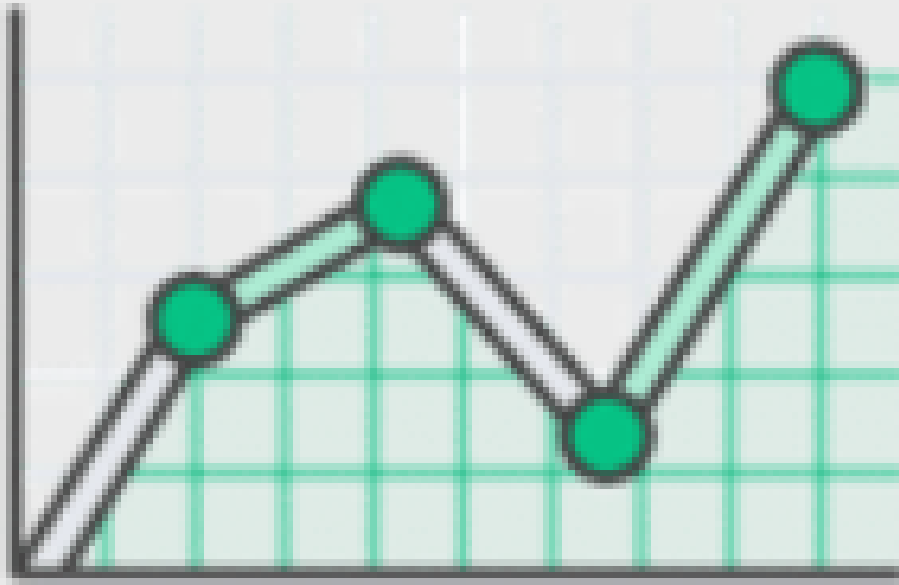
Rapid delivery

Support development by allowing teams to focus on innovation and product improvement. With a microservices architecture, development is decentralized, allowing each service to be built, deployed, and managed independently of the whole system. Automate the software release process with continuous integration and continuous delivery practices.



Reliability

Ensure the quality of application updates and infrastructure changes so you can reliably deliver at a more rapid pace while maintaining a positive experience for end users. Use practices like continuous integration and continuous delivery to test that each change is functional and safe. Monitoring and observability practices help you stay informed of performance in real-time.



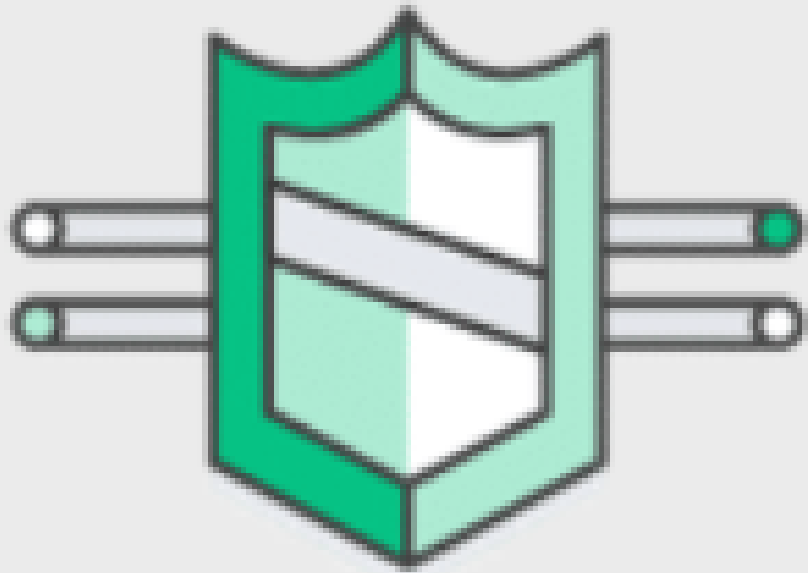
Scale

Operate and manage your infrastructure and development processes at scale. Automation and consistency help you manage complex or changing systems efficiently and with reduced risk. For example, infrastructure-as-code (IaC) helps you manage your development, testing, and production environments in a repeatable and more efficient manner.



Improved collaboration

Build more effective teams under a DevOps cultural model, which emphasizes values such as ownership and accountability. Developers and operations teams collaborate closely, share many responsibilities, and combine their workflows. This reduces inefficiencies and saves time. For example, writing code that takes into account the environment in which it is run, reduces handover periods between developers and operations.



Security

With DevOps, you can quickly while retaining control and preserving compliance. Use automated compliance policies, fine-grained controls, and configuration management techniques. Monitor for security incidents and adjust your configurations and software as needed. For example, using policy as code you can define and then track compliance at scale.

- Create a Highly collaborative environment
- Automate when possible
- Focus on customer needs
- Develop small and release often
- Include security at every phase
- Continuously experiment and learn
- Continuously improve

A CI/CD pipeline is a good example of how DevOps teams use tools to streamline workflows and standardize practices. A CI/CD pipeline assures code quality, security, and fast, consistent deployments by repeatably progressing through the pipeline.

DevOps teams iteratively remove process overlaps, human errors, and bottlenecks through automation.

Every DevOps team requires an efficient and reliable CI/CD pipeline. A CI/CD pipeline requires a well-integrated toolchain.

Communication and collaboration

Monitoring and observability

Continuous integration (CI)

Continuous delivery/continuous deployment (CD)

Microservices architecture

Infrastructure as code

DevOps practices require DevOps tools. Tools make processes easier, consistent, and predictable.

Cloud

Development

Examples:

- IDEs: AWS Cloud9, IntelliJ, Eclipse, Visual Studio Code
- SDKs: AWS SDK for Java, iPhone SDK
- Source code repositories: GitHub, AWS CodeCommit

CI/CD

Examples:

- Build tools: Jenkins, Travis CI, AWS CodeBuild
- Source control tools, repositories: Git, AWS CodeCommit
- Deployment tools: AWS CodeDeploy, AWS CloudFormation
- Pipeline automation tools: AWS CodePipeline, Jenkins, GitLab

Infrastructure automation

Examples:

- Infrastructure automation tools: AWS CloudFormation, Terraform, AWS Elastic Beanstalk
- Configuration management tools: Chef, Puppet, AWS OpsWorks

Containers and serverless

Examples:

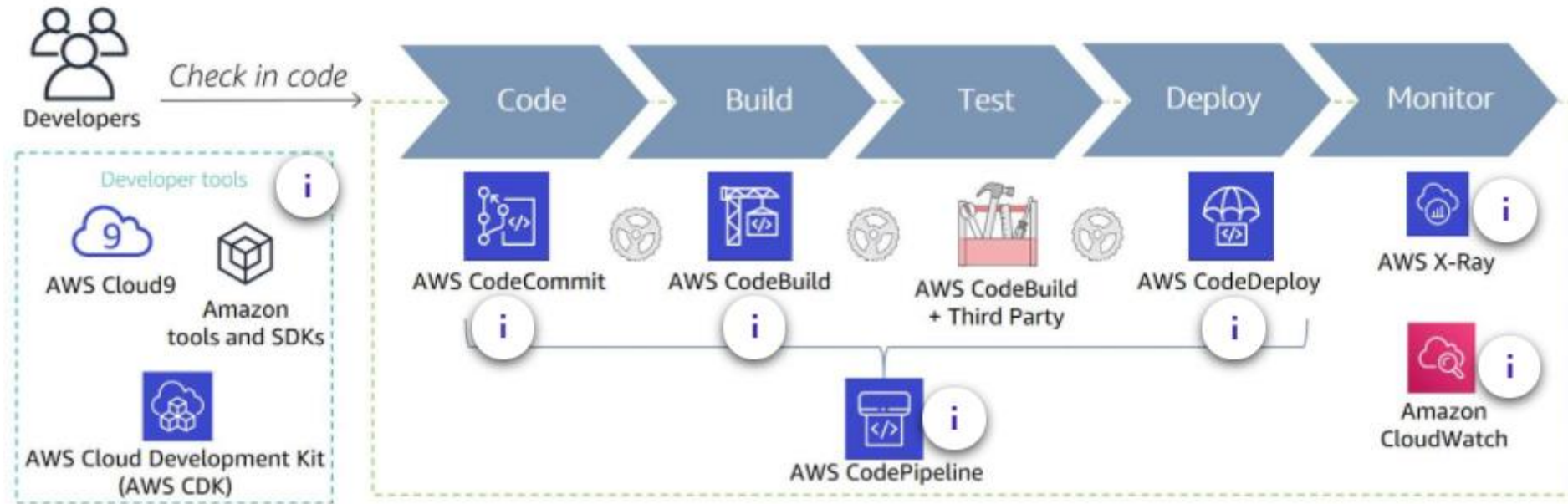
- Serverless services: AWS Lambda, AWS Fargate
- Container services:
 - Runtimes: Docker, Containerd
 - Orchestration: Amazon Elastic Container Service (Amazon ECS), Kubernetes, Amazon Elastic Kubernetes Service (Amazon EKS)

Monitoring and observability

Examples: AWS X-Ray, Amazon CloudWatch, AWS Config, AWS CloudTrail

Domain	% of Examination
Domain 1: SDLC Automation	22%
Domain 2: Configuration Management and Infrastructure as Code	19%
Domain 3: Monitoring and Logging	15%
Domain 4: Policies and Standards Automation	10%
Domain 5: Incident and Event Response	18%
Domain 6: High Availability, Fault Tolerance, and Disaster Recovery	16%
TOTAL	100%

- List the AWS services available to implement a successful DevOps methodology.
- Identify the AWS services used to automate the continuous integration and continuous delivery process.



Why use?

- Capture and visualize your pipeline, run it, view real-time status, and retry failed actions.
- Automate your release processes, eliminating human error, speed up delivery, and improve the quality of the release.
- Establish consistency in the release.
- Incorporate your source, build, and deploy tools.
- View pipeline history details.
- Integrate with third-party and AWS tools to build, test, and deploy your code when notified of a code change.

Monitoring

You can monitor your pipeline in a number of ways to assure its performance, reliability, and availability, and to find ways to improve it. You can monitor the pipeline directly from the AWS CodePipeline console, the command line interface (CLI), use Amazon EventBridge, or AWS CloudTrail.

Security

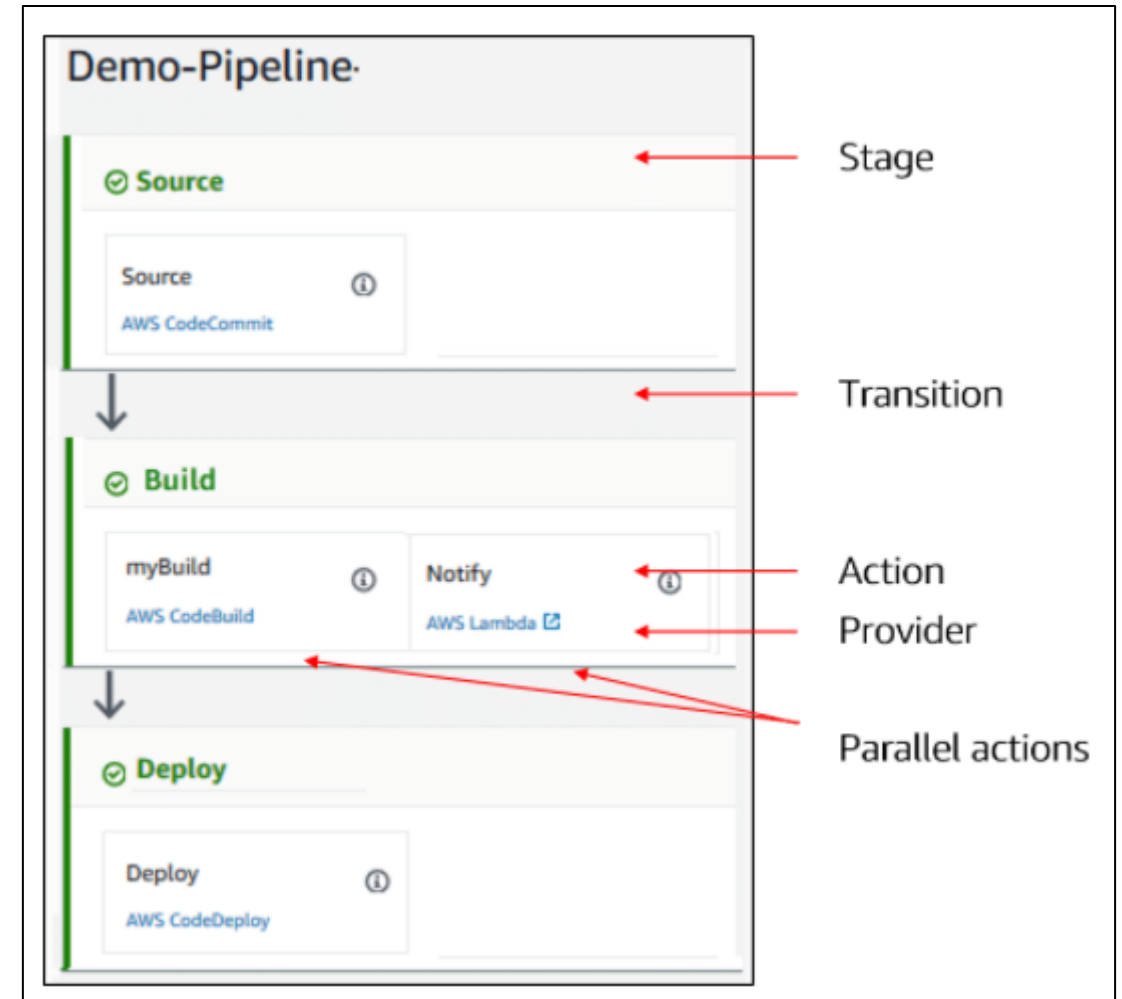
Security is an important part of any pipeline. CodePipeline supports resource-level permissions, enabling you to specify which user can perform what action on the pipeline. Some users might have read-only access to the pipeline, while others might have access to a particular stage or action within a stage. For more information about security, see [Security in AWS CodePipeline](#).

How it works

With CodePipeline, you model your release process with a number of stages and actions.

CodePipeline breaks up your release workflow into a series of stages, such as code, build, and test. Each stage can have a number of actions that need to be performed.

- Actions are tasks that can run in sequence or in parallel to each other. Actions are associated with a service provider that runs the action, or they can require user intervention. Service providers can be AWS services (like CodeBuild, Amazon Simple Storage Service (Amazon S3), AWS Lambda, and AWS CloudFormation), or third-party services (like Jenkins and TeamCity).
- Action types include:
 - Source (where the source is stored)
 - Build (how to build the application)
 - Test (how to test the application)
 - Deploy (how to deploy the application)
 - Approval (manual approval and notifications)
 - Invoke (Invoke a custom function).



Why use?

Eliminate the administrative overhead of managing your own hardware and software needed to store your code. CodeCommit is fully managed, highly available, and has no limits in the type or size of files it can store.

- Collaborate with your team on code using Git commands they already know. CodeCommit is a secure Git-based repository that can handle large numbers of files, code branches, and lengthy revision histories.
- Improve your existing workflow by integrating CodeCommit with other AWS services, IDEs, and third-party software (for example, Jira).

Monitoring

You can create notifications and trigger actions based on events such as creation of a branch, or when a commit is made.

Security

From a security perspective, access to the repository is managed with policies and user accounts. A version of the repository is found in the cloud and can be accessed through HTTPS or Secure Shell (SSH). The data is encrypted at rest and in transit. For more information about security, see [Security in AWS CodeCommit](#).

How it works

In CodeCommit, you create a repository. Users clone it to their environments, creating their own copy of the repository. After making changes, they push those changes back to the CodeCommit repository.

The CodeCommit repository is managed with Git source control. Git commands (such as `git add`, `git push`, `git branch`) can be used to work and collaborate on the code.

If it is part of a pipeline, CodeCommit can start the pipeline when a new code change is made on the configured CodeCommit repository and branch.

Below is a screen-capture of a CodeCommit repository named `myAppRepo`, and the files it contains. You are looking at the main branch. Notifications can be created and sent, based on what is happening in your repository. Note the Developer Tools panel which can be used to access other services, like CodeBuild, CodeDeploy, and CodePipeline.

Why use?

Eliminate the need to set up, patch, update, and manage your own build servers, since CodeBuild is fully managed.

- Automatically compile source code, run tests, and produce build artifacts.
- Specify build commands to run at each phase of the build.
- Process multiple builds concurrently, for example, developers can continuously build and test their code, catch errors early, and correct them early.
- Leverage out of the box preconfigured build environments (such as .NET Core, Java, Ruby, Python, Go, NodeJS, Android and Docker). Build environments contain the operating system, programming language runtime, and build tools (such as Apache Maven, Gradle). You can also provide custom build environments suited to your needs by means of Docker images.
- Pull source code from CodeCommit, Amazon S3, GitHub, GitHub Enterprise, and Bitbucket.
- Integrate CodeBuild with Jenkins to simplify the build process.

Monitoring

You can monitor the build through the CodeBuild console, CloudWatch Logs and several other ways.

Security

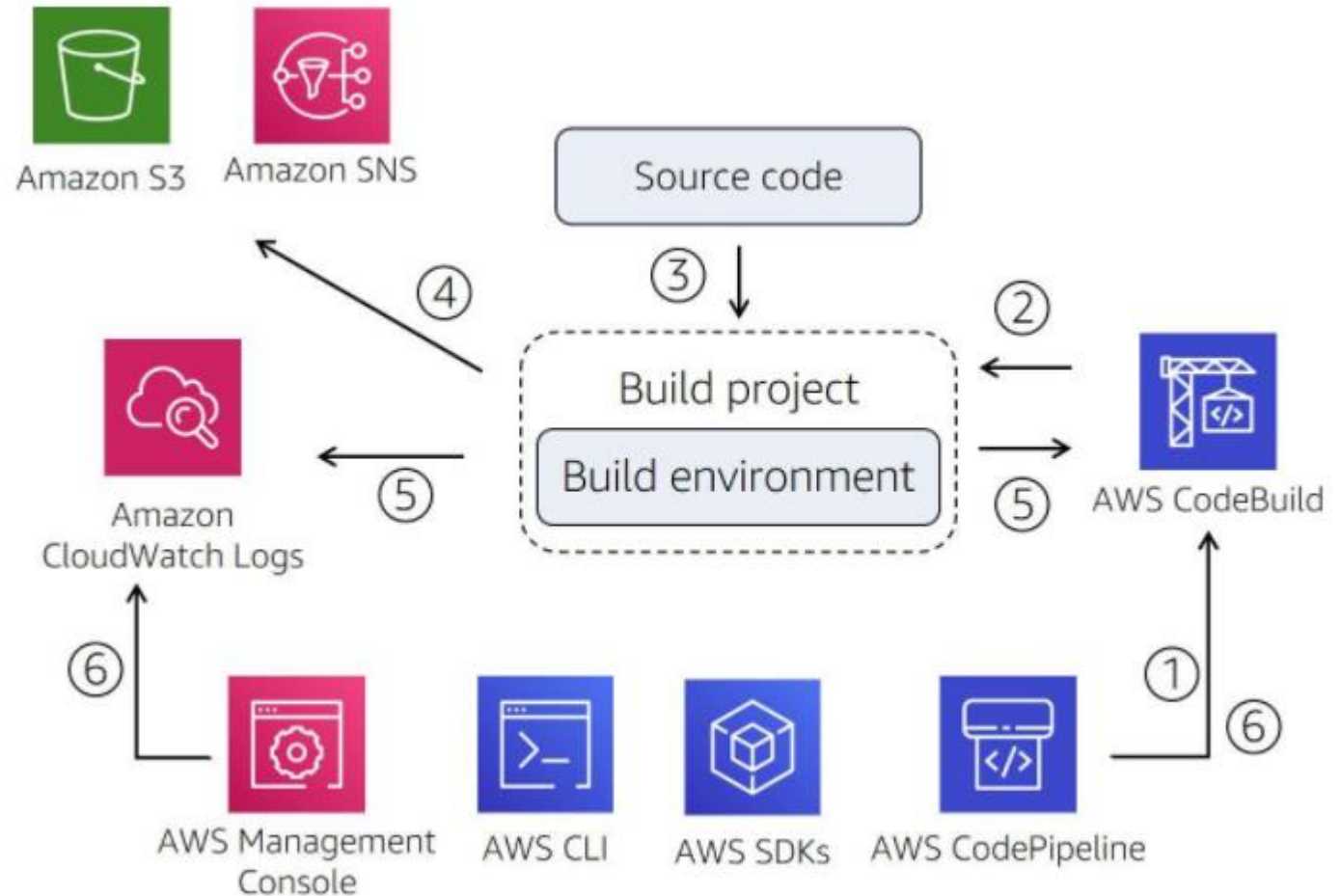
Build artifacts are encrypted and you control access to your build projects through resource-level permissions in AWS Identity and Access Management (IAM) policies. For more information about security, see [Security in AWS CodeBuild](#).

<https://aws.amazon.com/blogs/devops/test-reports-with-aws-codebuild/>

How it works

CodeBuild uses information provided by a build project to create a build environment (a provisioned Docker container created fresh on every build) where it then runs the build. Here is an overview of the steps that CodeBuild takes during the build.

1. CodeBuild uses a build project to run a build. A build project contains information needed to run the build, such as source repository location, runtime environment, build command, and where to store the build output. To access CodeBuild, use either the AWS Management Console, AWS Command Line Interface (AWS CLI), AWS SDKs, or CodePipeline.
2. CodeBuild uses the build project to create a build environment.
3. CodeBuild downloads the code and uses a build specification (buildspec) file to run a build. A buildspec is a collection of build commands that will run at each phase of the build (like installing tool packages, running tests, or packaging your code) and related settings.
4. Any build output is uploaded to an Amazon S3 bucket and able to configure notifications.
5. Build output is streamed to the service console, and CloudWatch Logs.
6. Monitor the progress of the build through CloudWatch or other services.



Why use?

Deploy server, serverless, or container applications.

- Automate deployments and eliminate the need for manual, error-prone operations. With CodeDeploy you can reliably and rapidly release new features and updates.
- Deploy on a variety of compute platforms including: AWS Lambda, Amazon ECS, Amazon EC2, or on-premises. You can even configure CodeDeploy to deploy to an Amazon EC2 Auto Scaling group, which will prepare the environment before traffic is sent to it.
- Concurrently deploy to one or multiple instances as the service scales to fit your needs.
- Minimize production downtime for your application by specifying if an update will be applied on an existing instance, or a newly provisioned environment that will replace the previous environment. You can also control how to handle the traffic-shifts from older to new versions. For example, if your application needs at least 50% of the instances in a deployment group to be up and serving traffic, you can specify that in your deployment configuration so that a deployment does not cause downtime.
- Automatically (or through user intervention) stop an unsuccessful deployment and roll back your deployment to a previous version.

Monitoring

AWS provides various tools that you can use to monitor CodeDeploy, including CloudWatch alarms, the CodeDeploy console and more.

Security

You can configure CodeDeploy to meet your security and compliance objectives. For more information on security, see [Security in AWS CodeDeploy](#).

How it works

To automate the deployment to the appropriate compute resources, CodeDeploy needs to know which files to copy, what scripts to run, and where to deploy.

The concept of an *application* is used by CodeDeploy to ensure it knows what to deploy (correct revision of code), where to deploy (deployment group), and how to deploy (deployment configuration).

Code

- Identify the correct version (revision) of the code.
- With the code, you provide an application specification file (AppSpec file) which is used to manage each deployment. During deployment, CodeDeploy looks for your AppSpec file in the root directory of the application's source.
- The AppSpec file specifies where to copy the code and how to get it running. For example, it tells CodeDeploy how to stop the application if it is already running, how to install the code, what command to run before and after the code is installed, and how to get the application running again.

Deployment configuration

- A *deployment configuration* is a set of deployment rules and deployment success and failure conditions used by AWS CodeDeploy during a deployment. For an Amazon EC2 compute platform, it specifies the number or percentage of instances that must remain available during deployment. It also specifies if an instance in the deployment group is briefly taken offline and updated with the latest code revision, or if a new instance replaces the instances in the deployment group.

Code

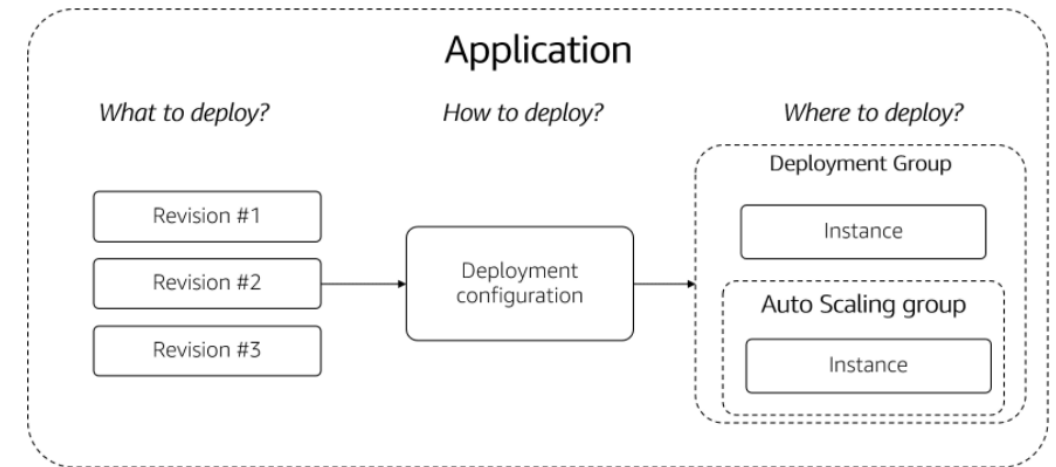
- Identify the correct version (revision) of the code.
- With the code, you provide an application specification file (AppSpec file) which is used to manage each deployment. During deployment, CodeDeploy looks for your AppSpec file in the root directory of the application's source.
- The AppSpec file specifies where to copy the code and how to get it running. For example, it tells CodeDeploy how to stop the application if it is already running, how to install the code, what command to run before and after the code is installed, and how to get the application running again.

Deployment configuration

- A *deployment configuration* is a set of deployment rules and deployment success and failure conditions used by AWS CodeDeploy during a deployment. For an Amazon EC2 compute platform, it specifies the number or percentage of instances that must remain available during deployment. It also specifies if an instance in the deployment group is briefly taken offline and updated with the latest code revision, or if a new instance replaces the instances in the deployment group.

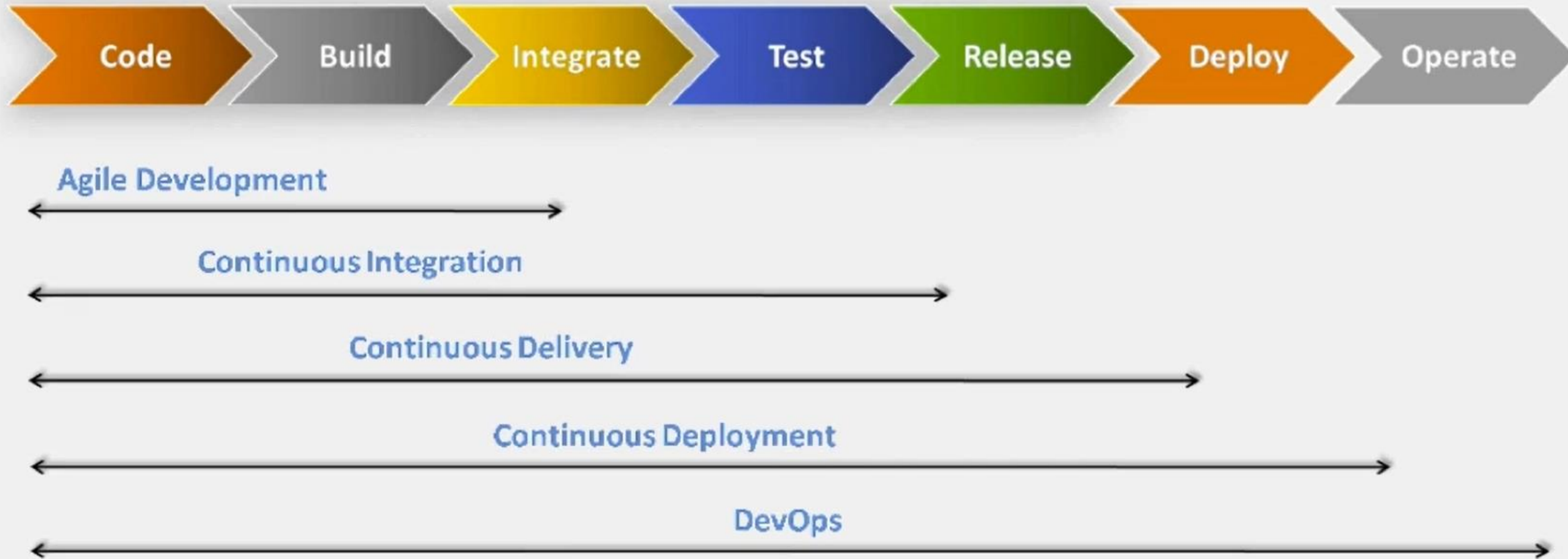
Deployment group

- A *deployment group* specifies the deployment targeted environment. The information it contains is specific to the target compute platform: AWS Lambda, Amazon ECS, Amazon EC2, or on-premises. For example, Amazon ECS lets you specify the Amazon ECS service, load balancer and more. For Amazon EC2, it is a logical group of deployment target instances or physical environments.
- A CodeDeploy application can have one or more deployment groups.
- Security needs to be assigned so the environment can communicate with CodeDeploy.
- The CodeDeploy agent is needed if you are deploying to Amazon EC2 or an on-premises compute platform. It is installed and configured on the target instances. It accepts and executes requests on behalf of CodeDeploy.



- Increased Speed
- Improved Collaboration
- Improved Security
- Rapid Delivery
- Scale Quickly
- Reliably deliver quality applications

- There is no need to host, maintain, and backup your own source control servers
- CodeCommit automatically encrypts your files in transit and at rest
- CodeCommit is built on highly available, redundant, and durable AWS services
- CodeCommit increases the speed and frequency of your development lifecycle by keeping repositories close to your builds



- Infrastructure as Code is the practice of managing applications infrastructure by describing and versioning it in code rather than through other manual methods
- Benefits
 - Cost Management
 - Reproducibility
 - Security
 - Consistency